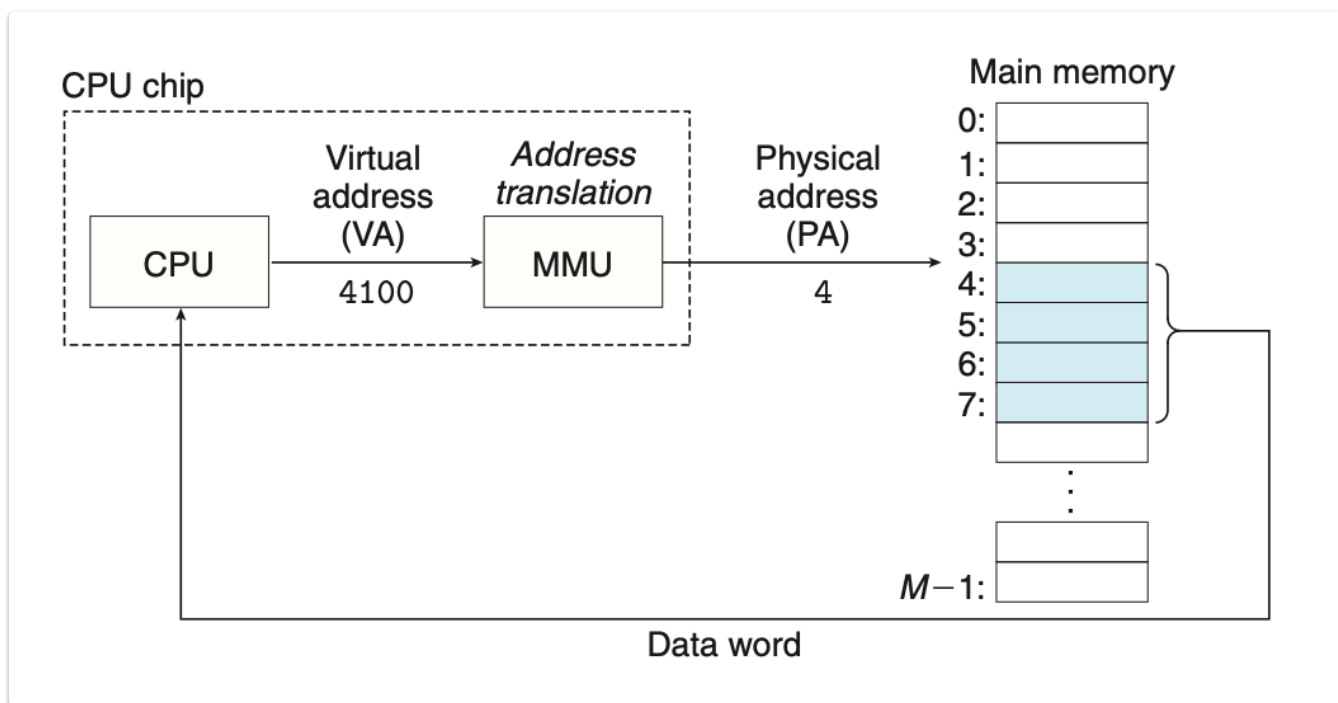


Physical and Virtual Addressing

最早期的电脑和一些嵌入式的处理器会使用物理寻址，现代的处理器的基本都会使用虚拟寻址，基本的架构如下图：



这种寻址方式需要 CPU 和 MMU (Memory Management Unit) 相互配合。MMU 会使用一个储存在主存中的查找表来进行地址翻译，这个查找表是由操作系统进行维护的。

Address Spaces

地址空间是一组非负整数地址的集合。

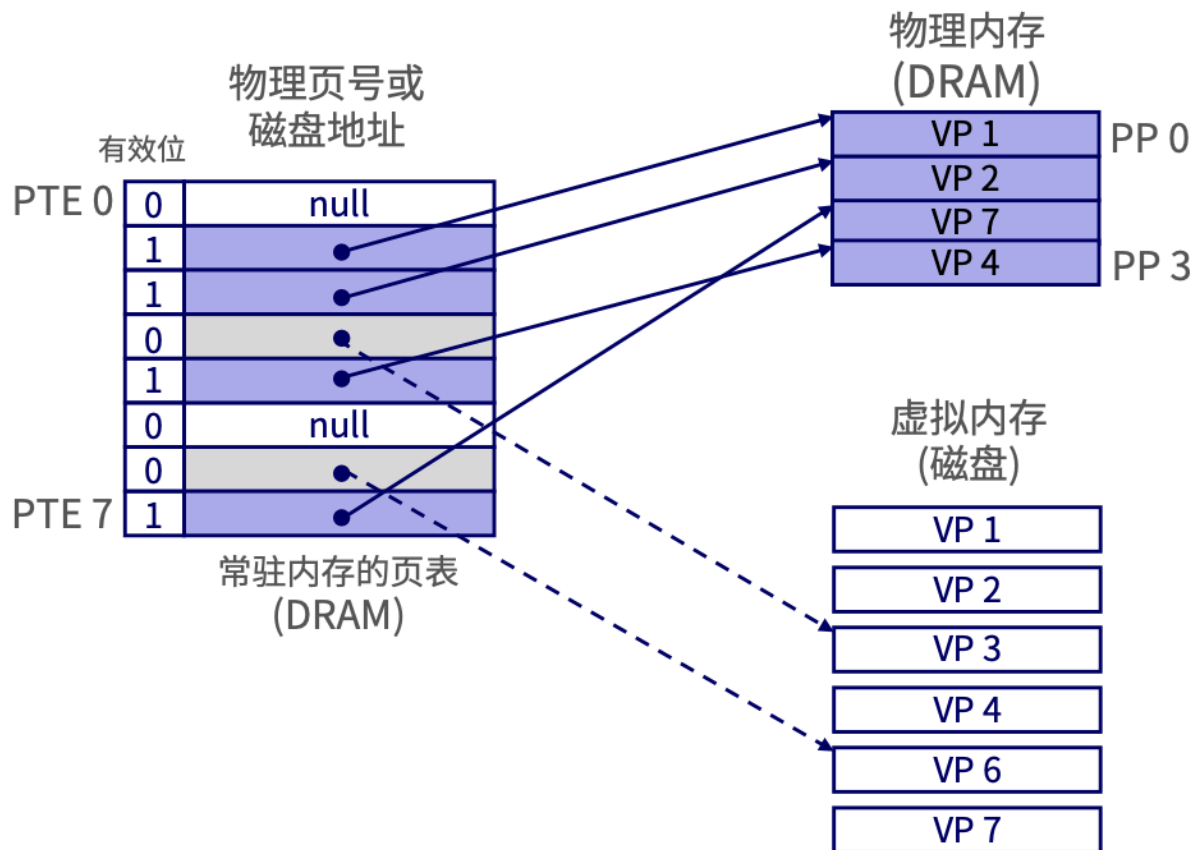
VM as a Tool for Caching

DRAM Cache Organization

概念上而言，虚拟内存被组织为一个由存放在磁盘上的 N 个连续的字节大小的单元组成的数组。这些内容，以页为单位，缓存在物理内存中 (DRAM cache)。

Page Table

页表用于将虚拟页地址翻译为物理页地址。页表可以看成 PTE 组成的数组。



接下来讨论一下不同阶段的命中/缺失处理

Page Hit

页命中：请求的地址的虚拟页对应的物理页存在于物理内存中，DRAM 缓存命中。

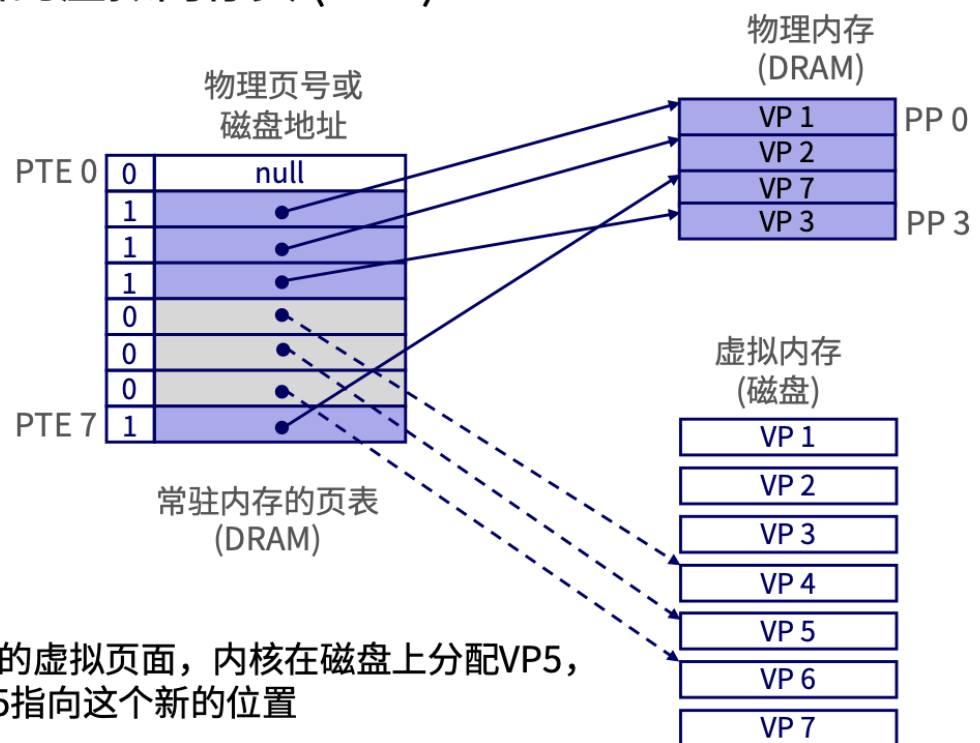
Page Fault

缺页：请求的地址的虚拟页对应的物理页不存在于物理内存中，DRAM 缓存不命中。此时需要：

- （可能）从物理内存中选择一个牺牲页驱逐
- 从磁盘读入目标页进入物理内存
- 重新启动导致缺页的指令

Allocating Pages

■ 分配一个新的虚拟内存页 (VP 5).



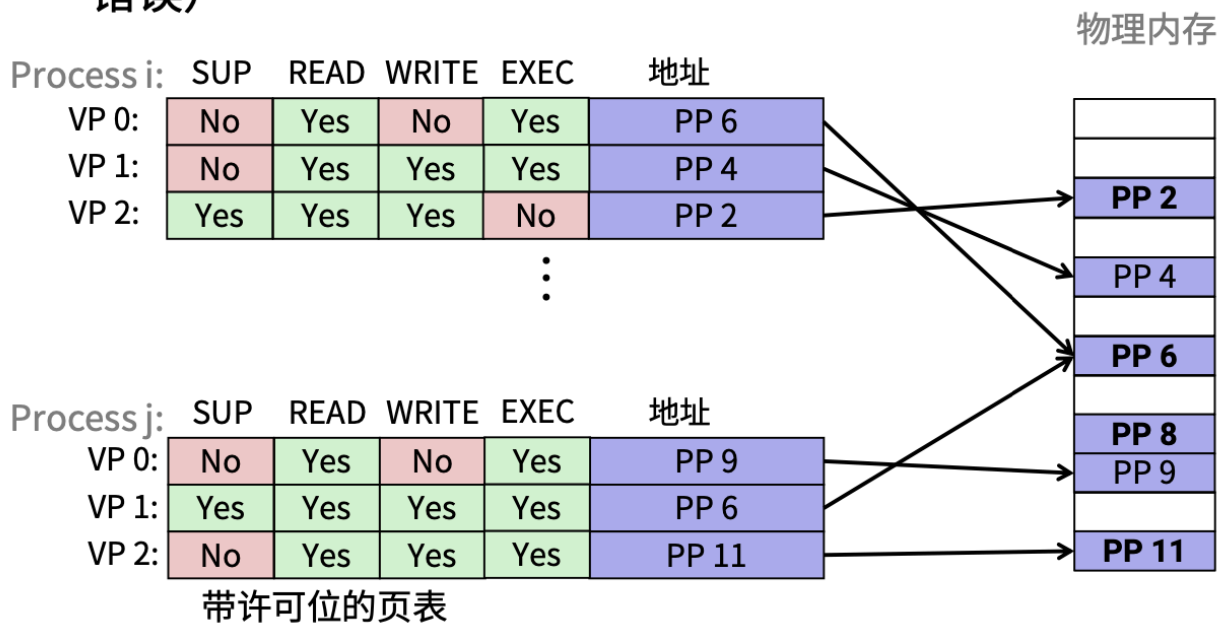
VM as a Tool for memory management

- 简化链接：每个程序使用相似的虚拟地址空间。代码、数据和堆都使用相同的起始地址。
- 简化加载：`execve` 为代码段和数据段分配虚拟页，并标记为无效（即未被缓存），实现延迟加载。每个页面被初次引用时，虚拟内存系统会按照需要自动地调入数据页。
- 简化共享：通过将不同的虚拟内存页面映射到相同的物理页面实现共享代码和数据
- 简化内存分配：分配连续的虚拟页时，无需分配连续的物理页，可以随意分配。

VM as a Tool for memory protection

通过在 PTE 上扩展许可位，可以保存该页的权限信息（是否 S 态、是否可读、是否可写、是否可执行）

- 在 PTE 上扩展许可位以提供更好的访问控制
- 内存管理单元 (MMU) 每次访问数据都要检查许可位 (段错误)



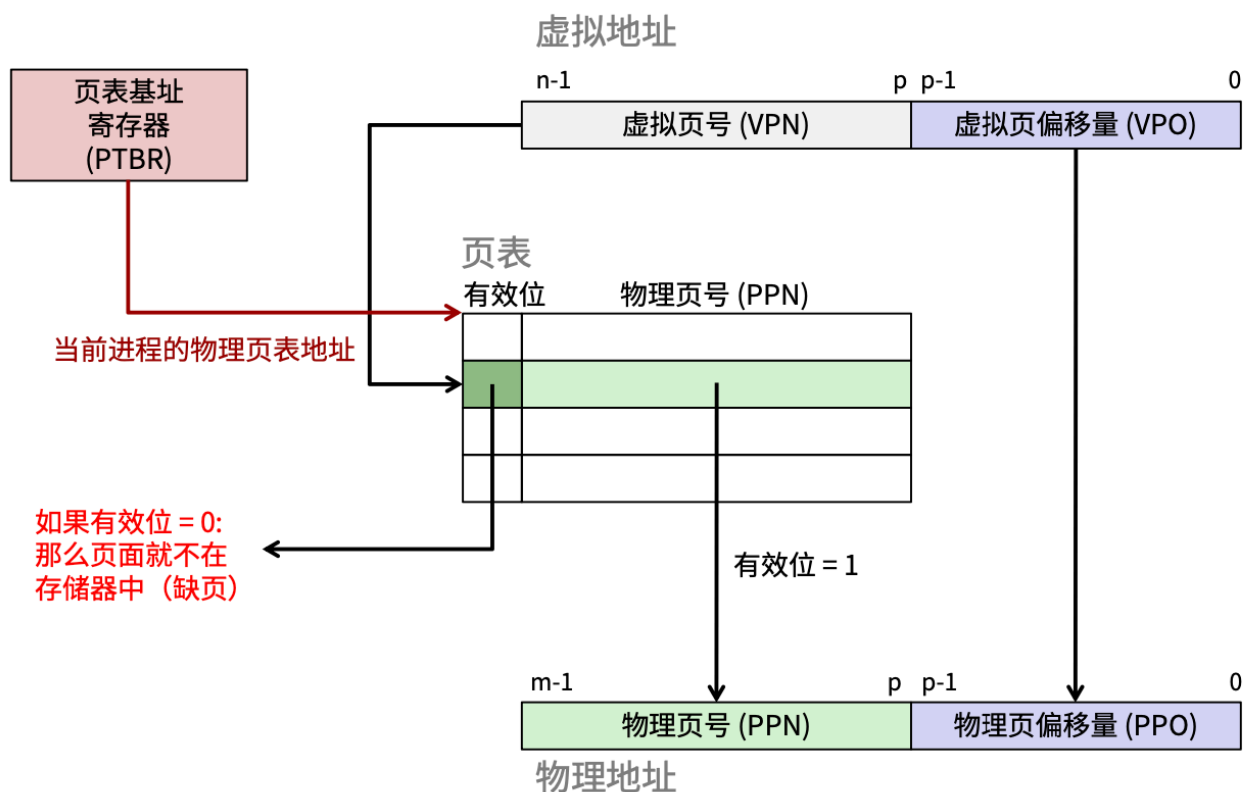
Address translation

地址翻译使用到的所有符号

- Basic Parameters 基本参数
 - $N = 2^n$: 虚拟地址空间中的地址数量
 - $M = 2^m$: 物理地址空间中的地址数量
 - $P = 2^p$: 页的大小 (bytes)
- Components of the virtual address (VA) 虚拟地址组成部分
 - TLBI: TLB index----TLB索引 (翻译后备缓冲器索引)
 - TLBT: TLB tag----TLB标记
 - VPO: Virtual page offset----虚拟页面偏移量 (字节)
 - VPN: Virtual page number----虚拟页号
- Components of the physical address (PA) 物理地址组成部分
 - PPO: Physical page offset (same as VPO)----物理页面偏移量
 - PPN: Physical page number----物理页号

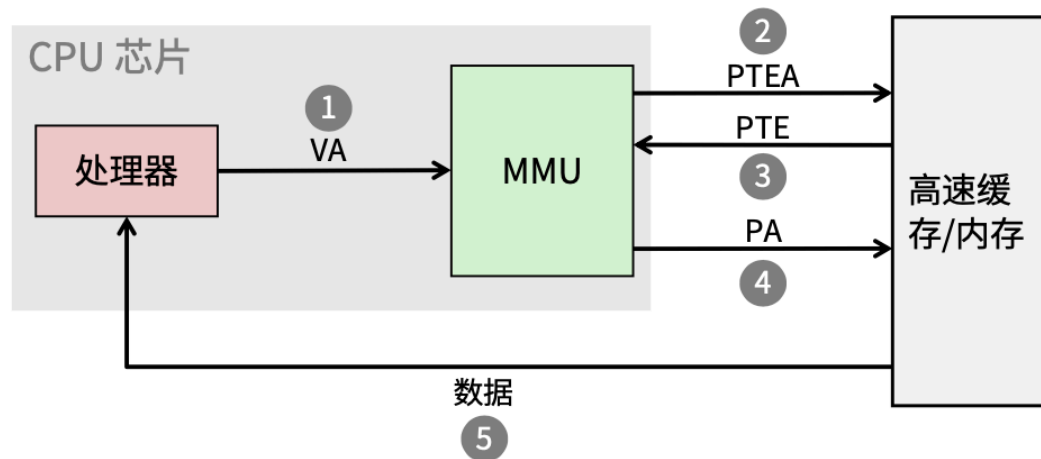
Address Translation With a Page Table

基于页表的地址翻译



Address Translation: Page Hit

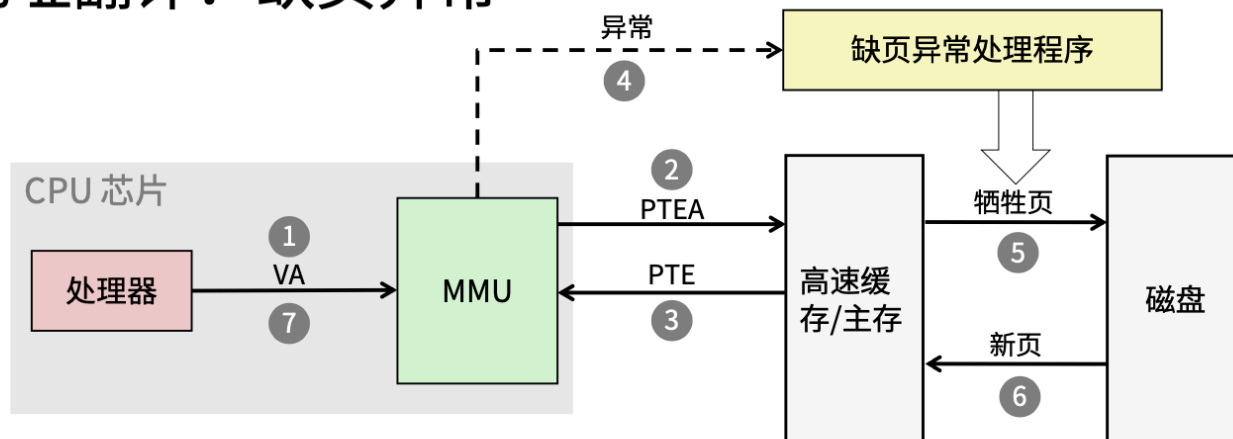
地址翻译：页面命中



- 1) 处理器生成一个虚拟地址，并将其传送给MMU
- 2-3) MMU 使用内存中的页表生成PTE地址
- 4) MMU 将物理地址传送给高速缓存/主存
- 5) 高速缓存/主存返回所请求的数据字给处理器

Address Translation: Page Fault

地址翻译：缺页异常



1) 处理器将虚拟地址发送给 MMU

2-3) MMU 使用内存中的页表生成PTE地址

4) 有效位为零, 因此 MMU 触发缺页异常

5) 缺页处理程序确定物理内存中替换页 (若页面被修改, 则换出到磁盘)

6) 缺页处理程序调入新的页面, 并更新内存中的PTE

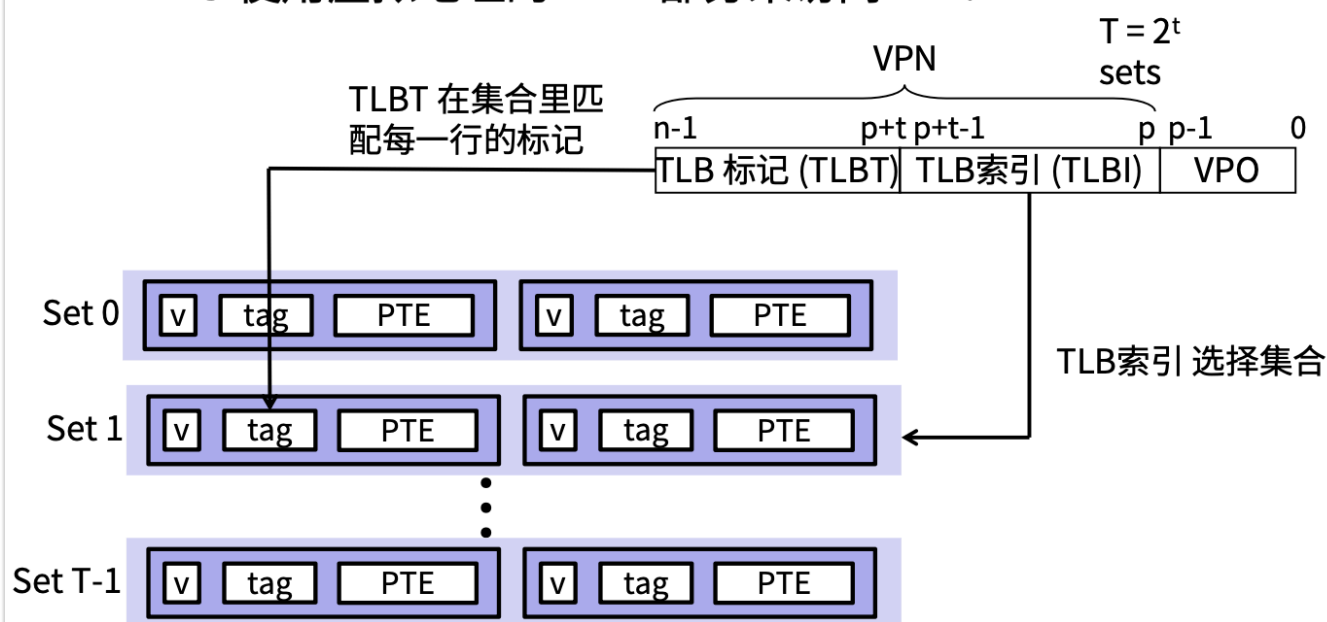
7) 缺页处理程序返回到原来进程, 再次执行缺页的指令

3

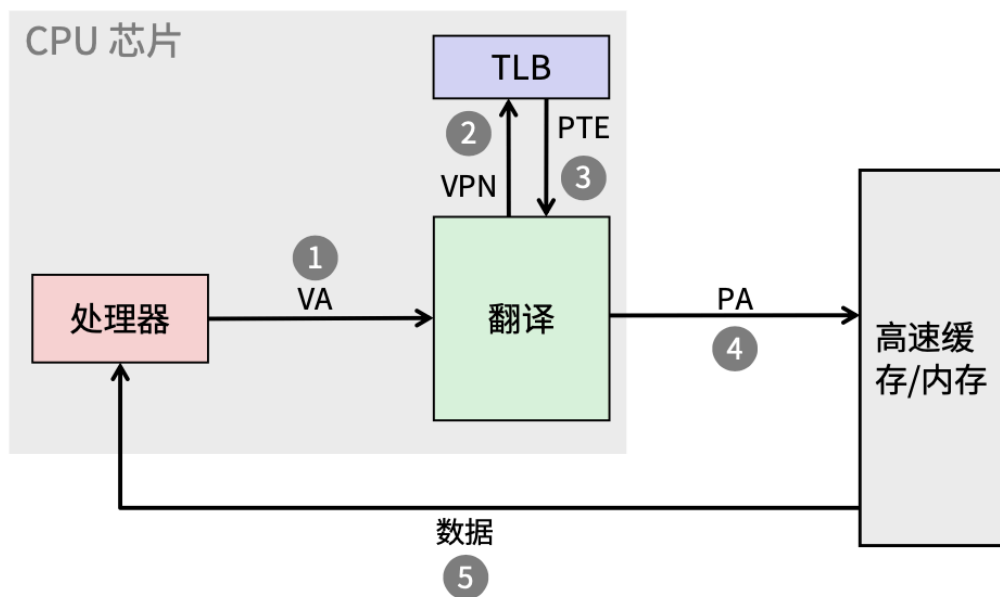
TLB

前面的 PTE 似乎只是恰巧缓存在 L1, 仍有可能被其他数据引用驱逐。即使命中也会带来延迟, 我们希望在 L1 和 MMU 之间再加入一个存储层次, 以缓存 PTE。解决办法: 加入 TLB

■ MMU 使用虚拟地址的 VPN 部分来访问TLB:

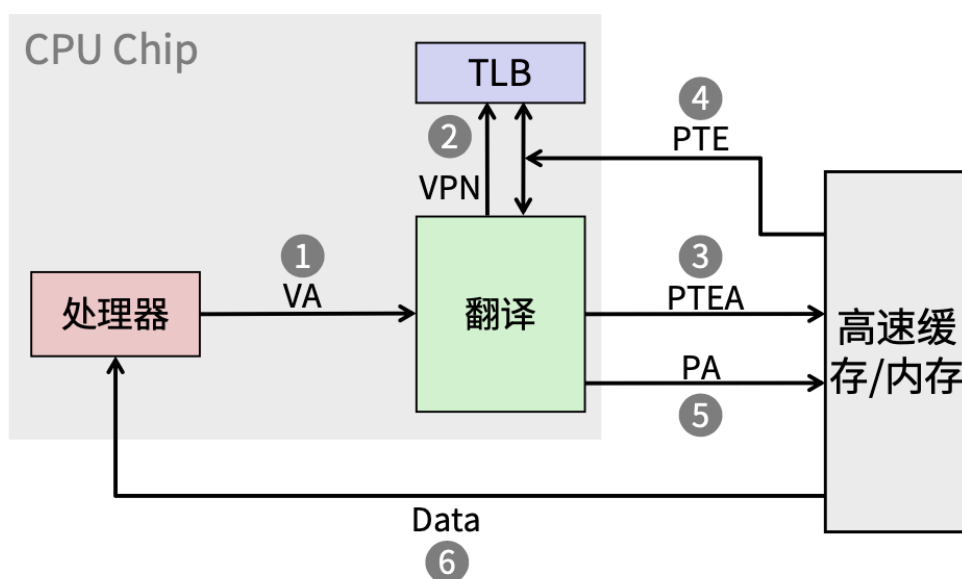


TLB Hit TLB命中



TLB 命中减少内存访问

TLB 不命中

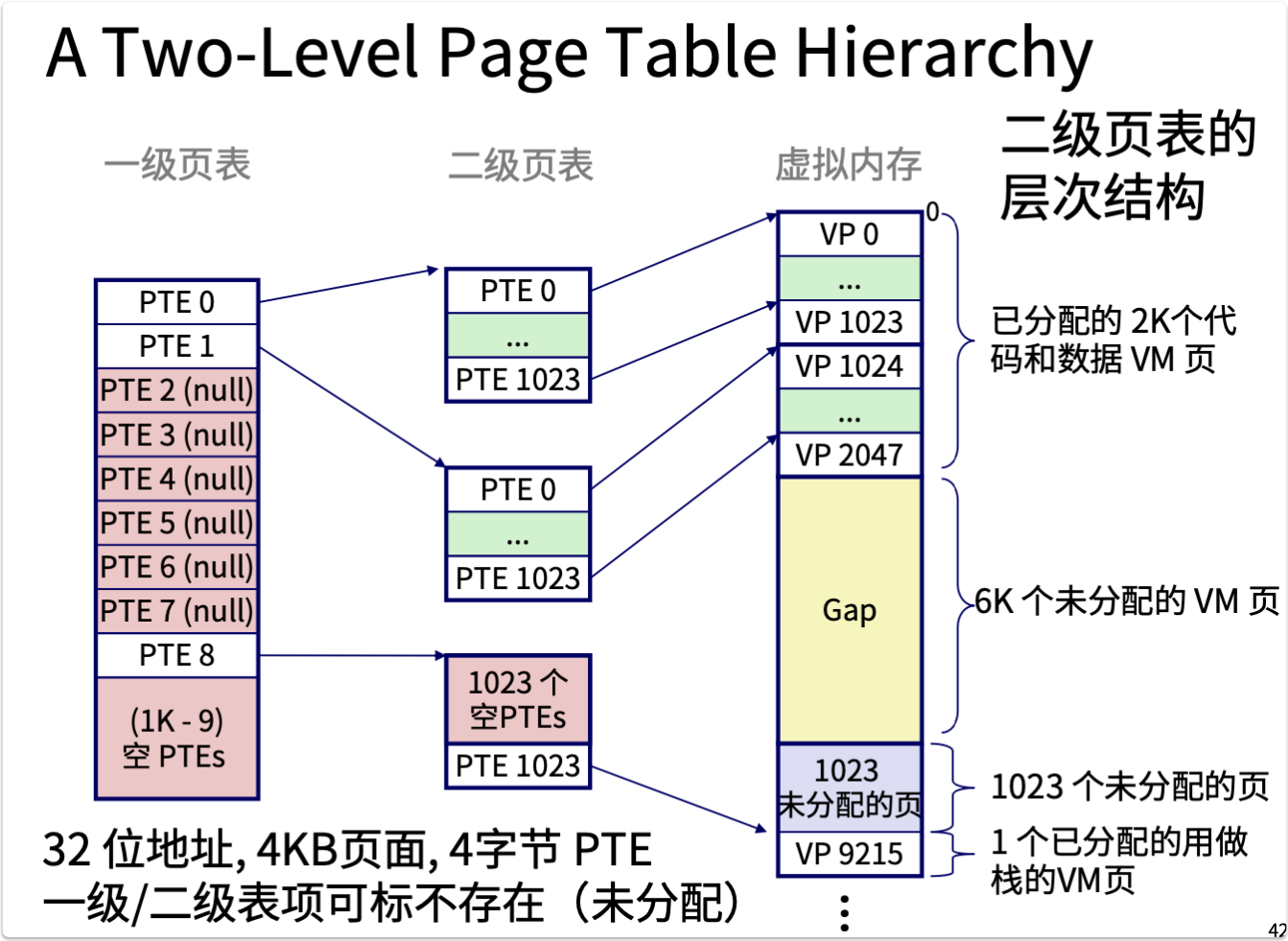


TLB 不命中引发了额外的内存访问

万幸的是, TLB 不命中很少发生。这是为什么呢? --局部性

TLB 条目数一般在 64-1024 之间。由于程序地址访问存在局部性, 所以命中率会很可观。

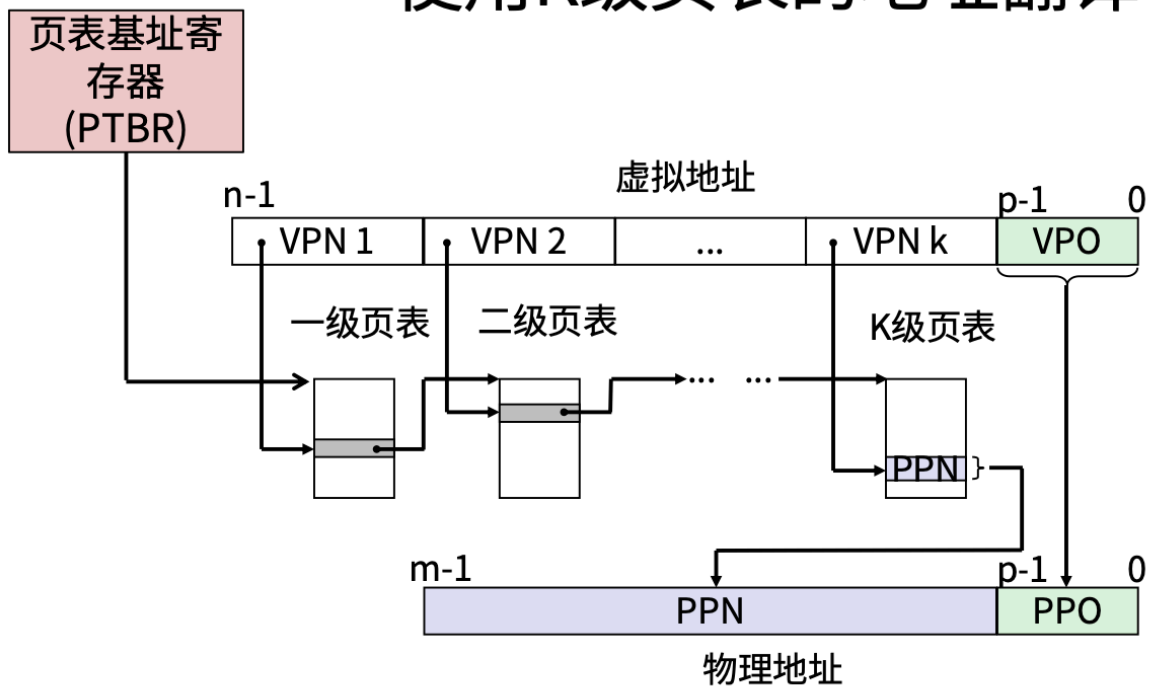
多级页表可以缩小页表的大小。



如上图，原来需要 1024 个 PTE 代表的未分配区域，现在只需要一个 PTE 就可以代表。

Translating with a k-level Page Table

使用K级页表的地址翻译



内存系统示例

(往年题都考了， 比较重要)

Review of Symbols符号回顾

■ 基本参数

- $N = 2^n$: 虚拟地址空间中的地址数量
- $M = 2^m$: 物理地址空间中的地址数量
- $P = 2^p$: 页的大小 (bytes)

■ 虚拟地址组成部分

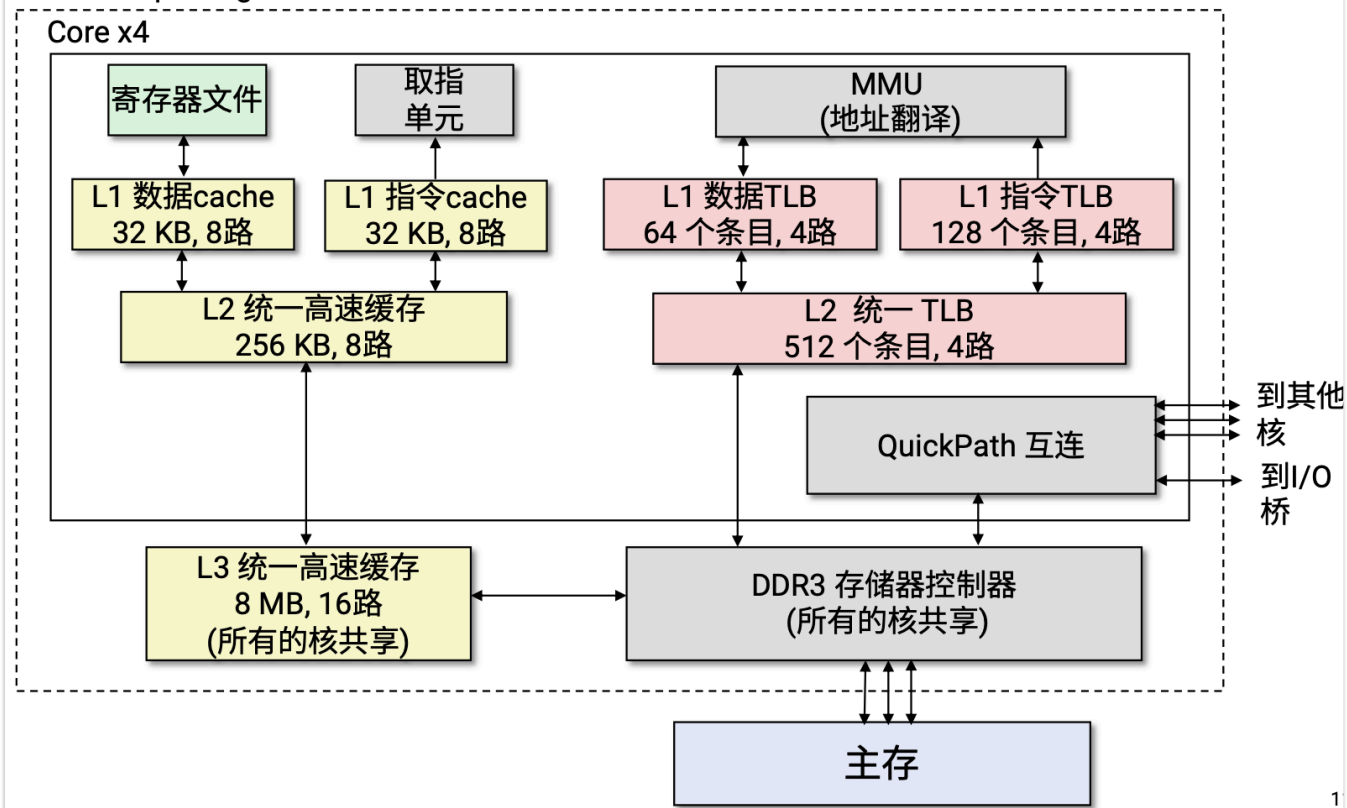
- TLBI: TLB索引
- TLBT: TLB 标记
- VPO: 虚拟页面偏移量 (字节)
- VPN: 虚拟页号

■ 物理地址组成部分

- PPO: 物理页面偏移量 (same as VPO)
- PPN: 物理页号
- CO: 缓冲块内的字节偏移量
- CI: Cache 索引
- CT: Cache 标记

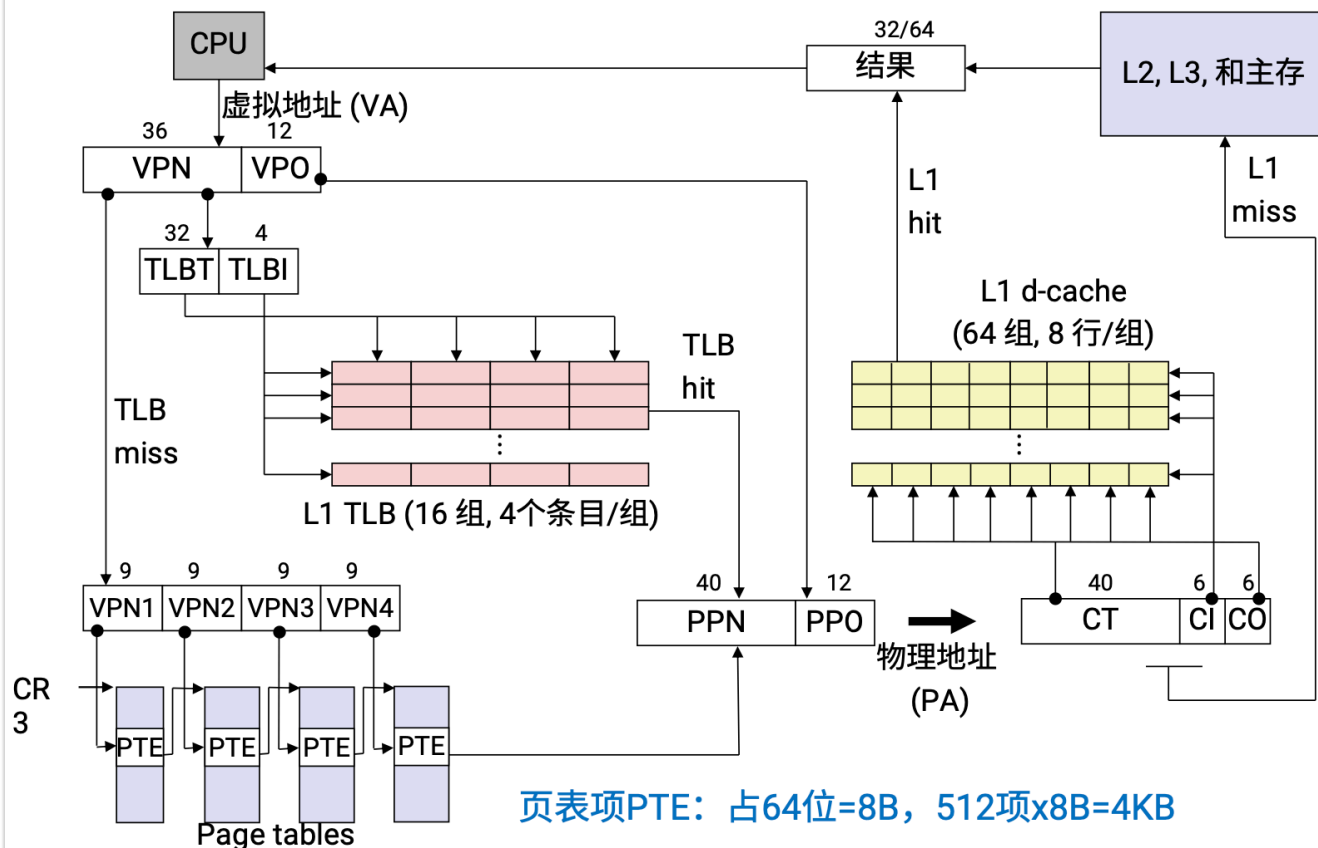
Intel Core i7 内存系统

Processor package



注意 L1 cache 用的都是物理地址，需要通过 mmu 将虚拟地址翻译得到。

Core i7 地址翻译 (VA48位PA52位)



页表项PTE: 占64位=8B, 512项x8B=4KB

TLBT/TLBI: 条目/路数 = 组数, 即可得到。

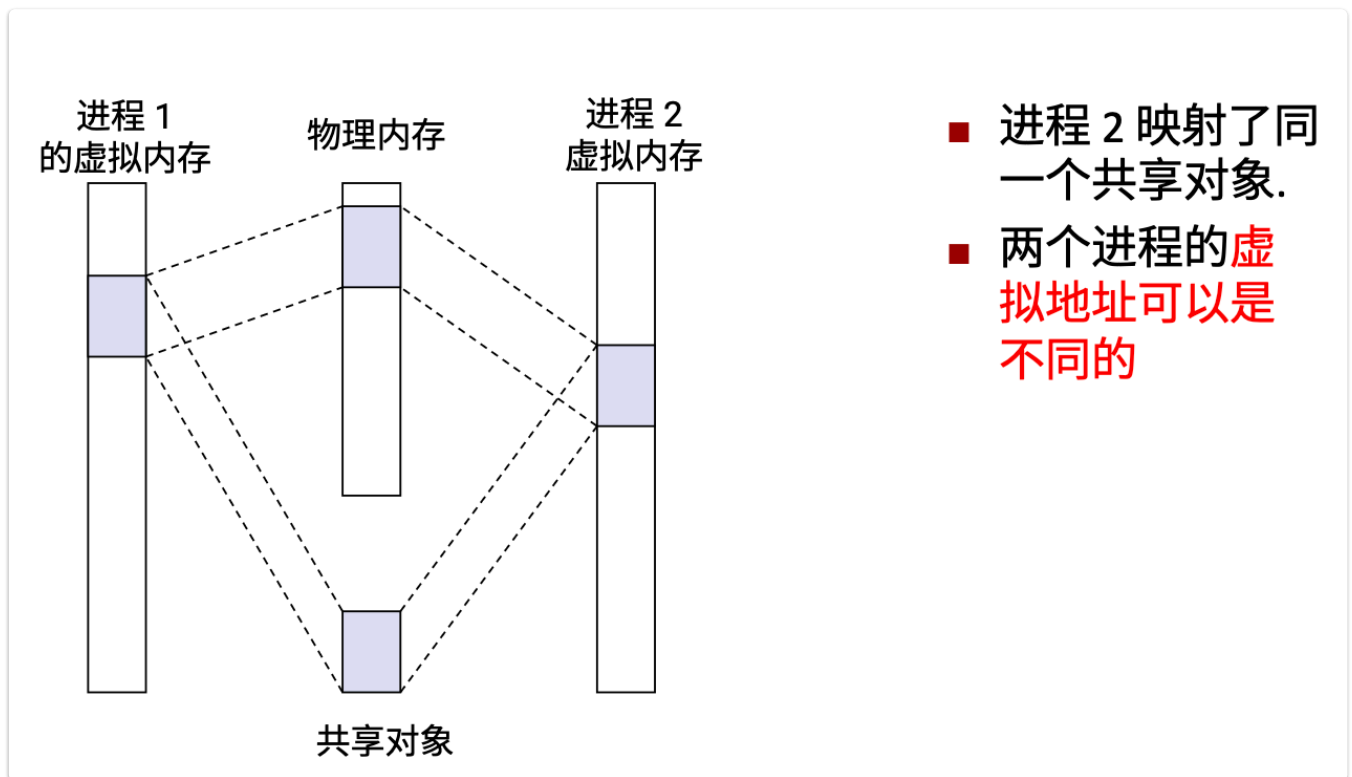
CT/CI/CO: 先计算总块数, 总块数/路数 = 组数, 剩余易得。

内存映射

内存映射是一个宽泛的概念, 指虚拟内存区域与磁盘上的对象关联起来以初始化这个虚拟内存区域的内容。虚拟内存区域可以映射: 普通文件、匿名文件等。

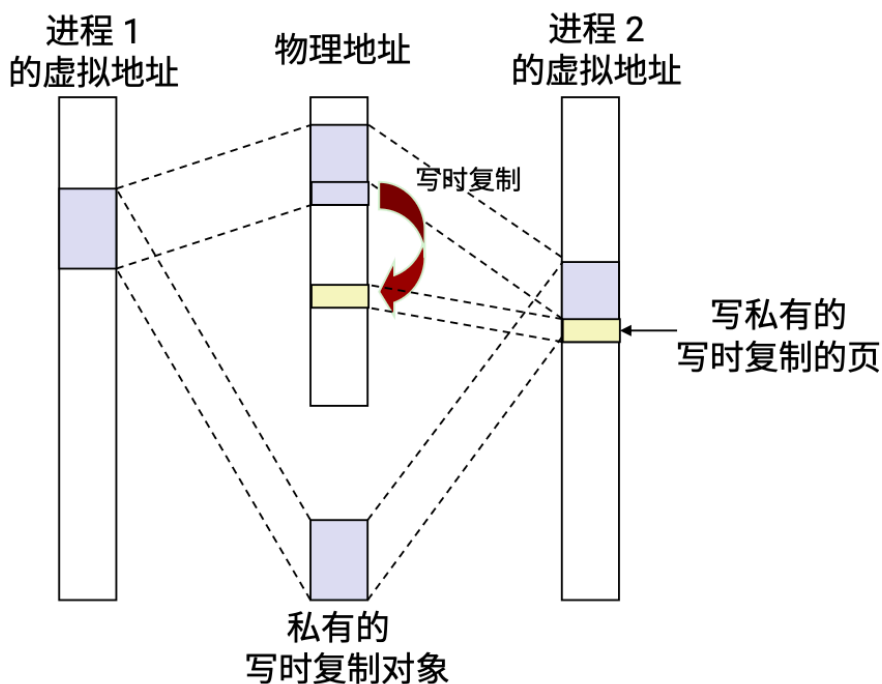
共享对象

不同的进程可以共享对象, 只需要将对应的虚拟内存映射到相同物理内存即可



对于堆栈、数据、堆区、寄存器私有的写时复制 (Copy-on-write) 对象:

共享对象： 私有的写时复制（Copy-on-write）对象



- 写私有页的指令触发保护故障
- 故障处理程序创建这个页面的一个新副本，更新PTE条目，且可写
- 故障处理程序返回时重新执行写指令
- 尽可能地延迟拷贝（创建副本）充分利用物理内存

用户级内存映射

使用 `mmap` 函数/系统调用，可以将磁盘文件映射到虚拟内存区域。